Newton School of Technology

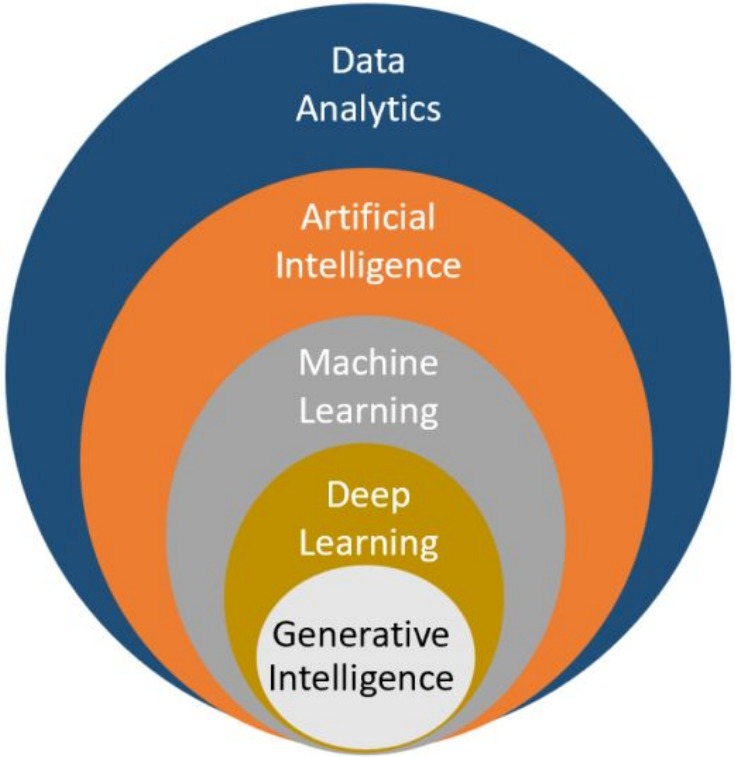
Join the lecture online on your dashboard.

Deep Learning Foundations

Mathematical Intuition & Architecture

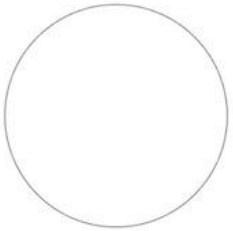
Setting the context

AI, ML, DL and GI – How it all fits together!

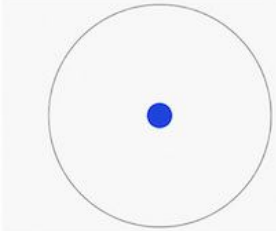


Source: Anang B Singh, 20231010

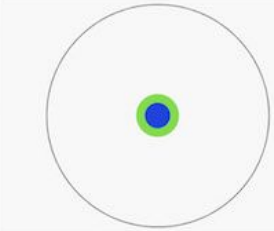
Imagine a circle that contains all of human knowledge:



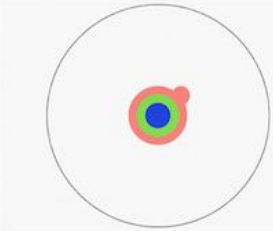
By the time you finish elementary school, you know a little:



By the time you finish high school, you know a bit more:

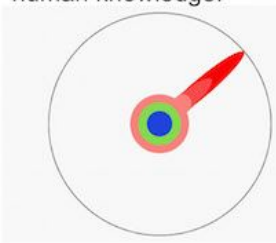


With a bachelor's degree, you gain a specialty:

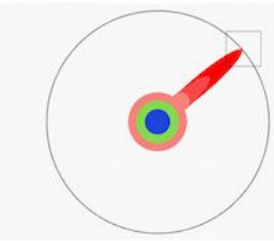


A master's degree deepens that specialty:

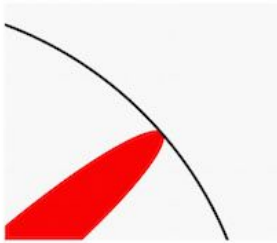
Reading research papers takes you to the edge of human knowledge:



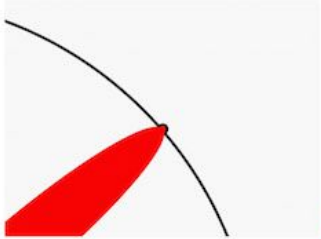
Once you're at the boundary, you focus:



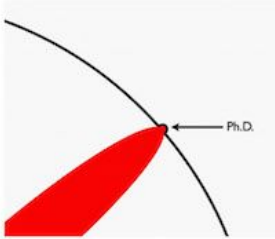
You push at the boundary for a few years:



Until one day, the boundary gives way:

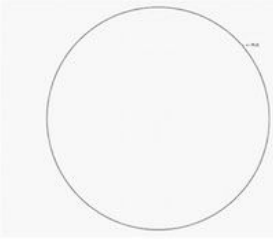


And, that dent you've made is called a Ph.D.:



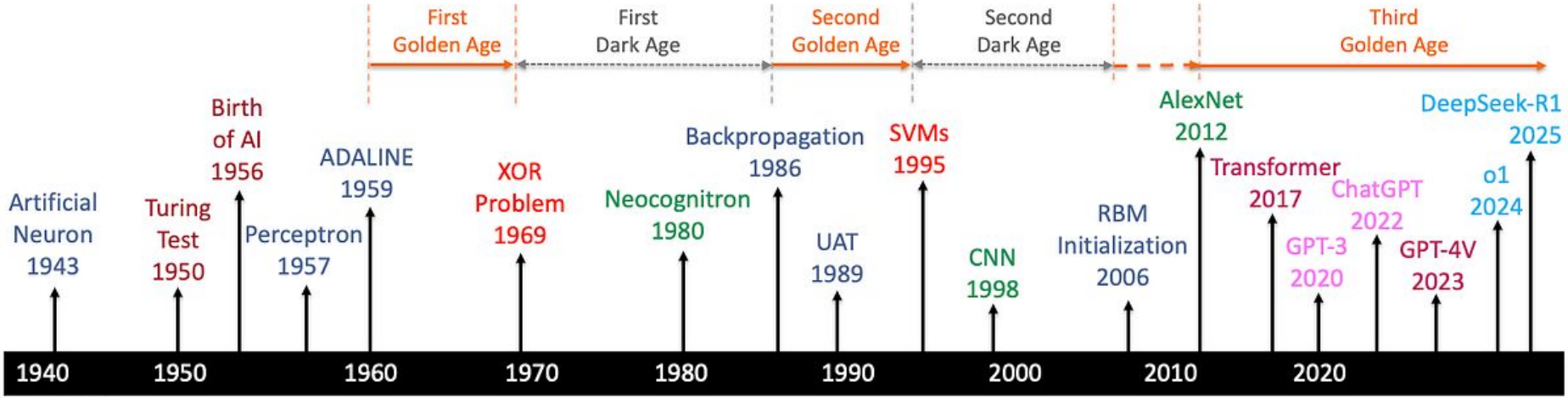
Of course, the world looks different to you now:

So, don't forget the bigger picture:



Keep pushing.

A Brief History of AI with Deep Learning



McCulloch-Pitts

Rosenblatt

Widrow-Hoff

Minsky-Papert

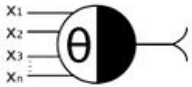
Rumelhart, Hinton et al.

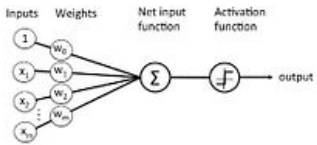
LeCun

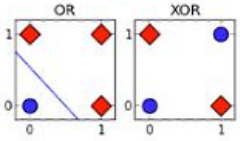
Hinton-Ruslan

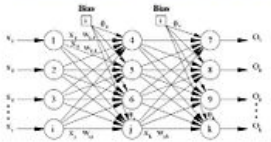
Krizhevsky et al.

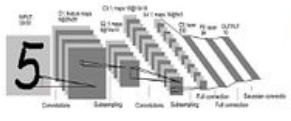
Vaswani

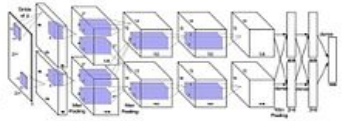


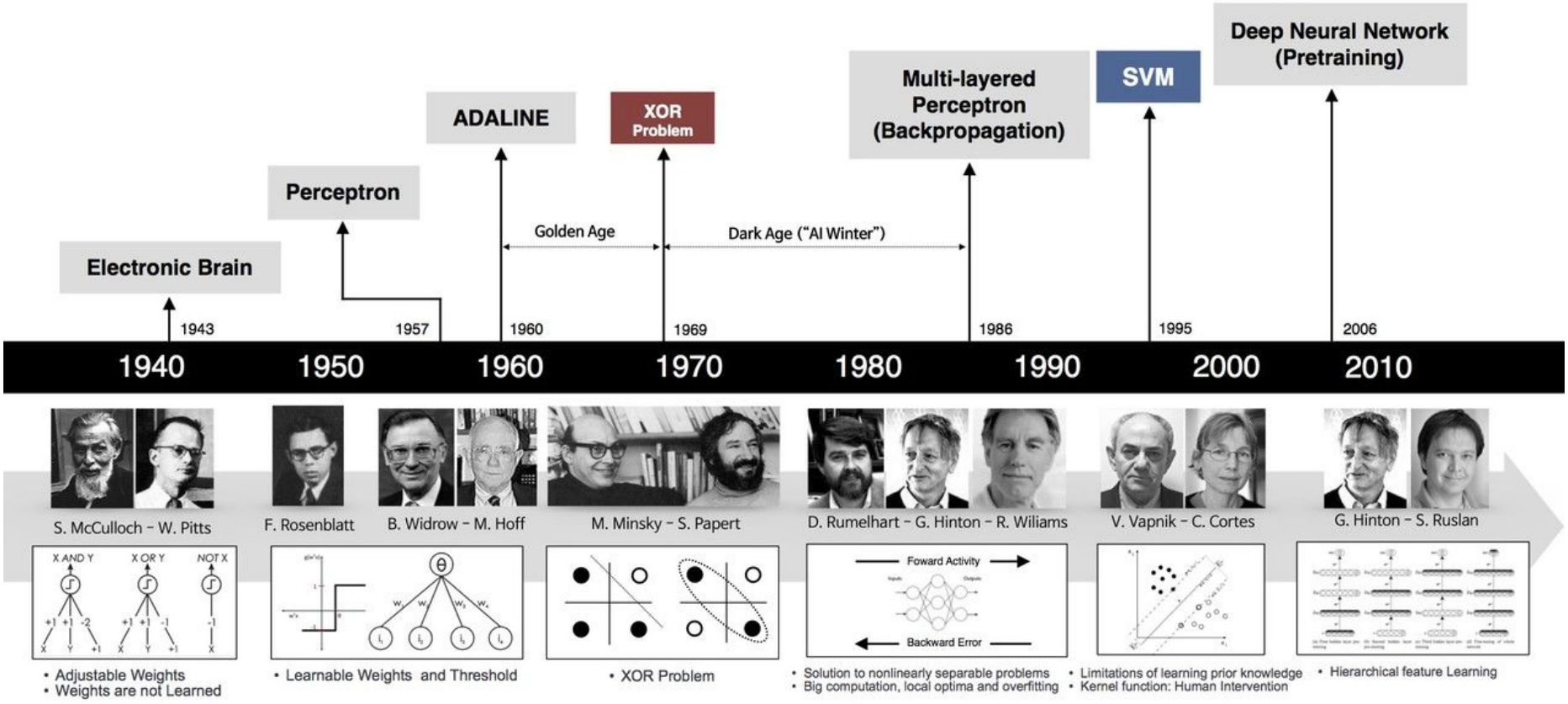











Deep Neural Network (Pretraining)
SVM
Multi-layered Perceptron (Backpropagation)
XOR Problem
ADALINE
Perceptron
Electronic Brain
Golden Age
Dark Age ("AI Winter")
1943
1957
1960
1969
1986
1995
2006
1940
1950
1960
1970
1980
1990
2000
2010
S. McCulloch - W. Pitts
F. Rosenblatt
B. Widrow - M. Hoff
M. Minsky - S. Papert
D. Rumelhart - G. Hinton - R. Wiliams
V. Vapnik - C. Cortes
G. Hinton - S. Ruslan
X AND Y
X OR Y
NOT X
Foward Activity
Backward Error
• Adjustable Weights
• Weights are not Learned
• Learnable Weights and Threshold
• XOR Problem
• Solution to nonlinearly separable problems
• Big computation, local optima and overfitting
• Limitations of learning prior knowledge
• Kernel function: Human Intervention
• Hierarchical feature Learning

Please explore the document attached in the slack
Deep learning resources

And the pre-read/pre-watch, it can help with the lecture understanding. This is a heavy course.

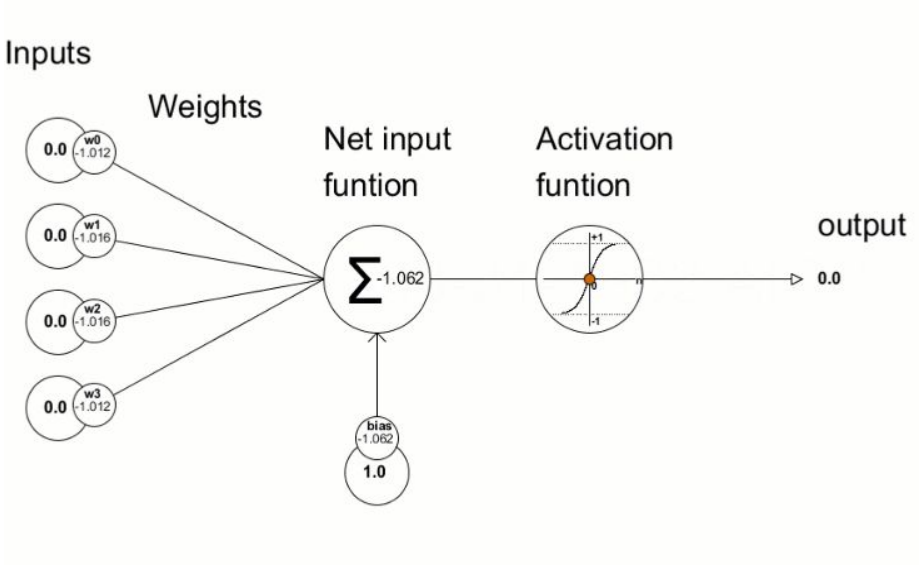
Playlist for today (3blue 1 brown)
Neural networks - YouTube

The basics

Perceptron vs Regression

Deep learning networks learn from data by making mistakes.

At the heart of this learning process lie the perceptron, activation functions, and the backpropagation algorithm.



$$\text{Output, } y = \phi\left(\sum_{i=1}^{n} w_i x_i + b\right)$$

- x_i: Input features
- w_i: Weights associated with each input
- b: Bias term
- ϕ: Activation function

The Limit of Linearity

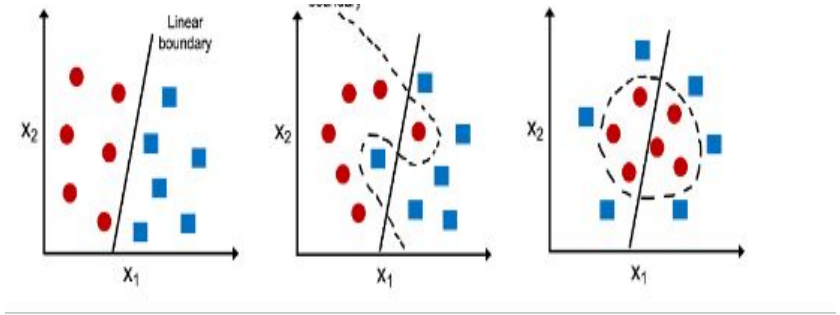
Newton School of Technology

The Concept: A dataset is "linearly separable" if a single straight line (or hyperplane) can perfectly separate the classes.

Success Case: AND and OR logic gates *are* linearly separable. A simple perceptron can solve them.

Failure Case: Most real-world data is complex and intertwined, requiring curved or complex decision boundaries.

The Constraint: No matter how long you train a linear model, it cannot solve a non-linear problem.



The XOR Problem Defined

Newton School of Technology

Input x_1	Input x_2	Output y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Logical Intuition: It's not just "either/or". It is "one or the other, but *not* both".

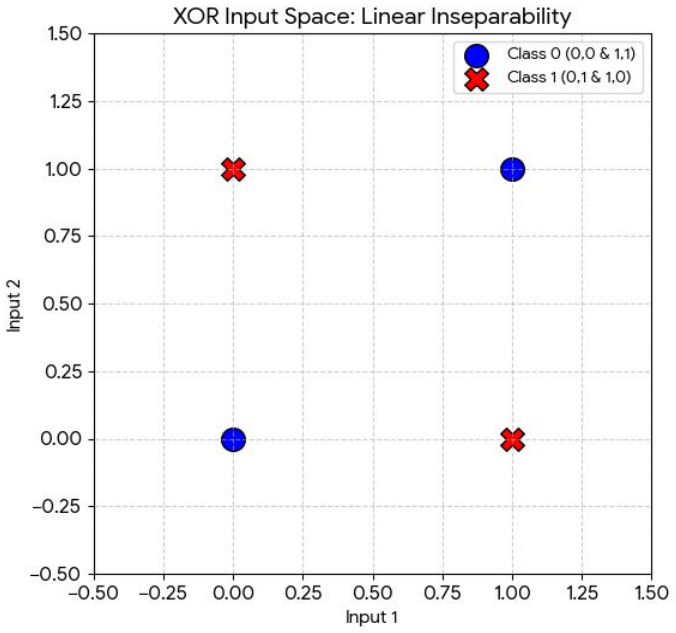
Visualizing the XOR Impasse

The **positive examples** are at (0,1) and (1,0).

The **negative examples** are at (0,0) and (1,1).

Try drawing a line: Any straight line that separates the 0s from the 1s is impossible to draw.

You need ***two*** **lines, or a curved boundary,** to separate these classes.



What to do now?

Ditch it...duh

At-least that's what we did _(T_T)_/

The Solution: Hidden Layers

Newton School of Technology

Expanding the Feature Space

To solve XOR, we introduce a **Hidden Layer** between the input and output.

The hidden layer transforms the input space into a new representation.

Feature Transformation: The hidden neurons learn intermediate features (e.g., OR and NAND functions).

In this new feature space, the data becomes linearly separable for the final output layer.

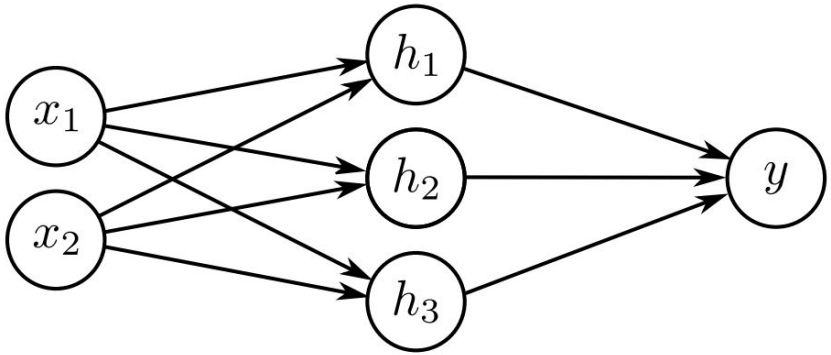
Shallow Neural Network

Newton School of Technology

Input Layer: Receives the raw data vector (e.g., pixel intensities). No computation happens here.

Hidden Layer: The "deep" part. They perform intermediate computations and feature extraction. Can be one or many. One for **shallow**.

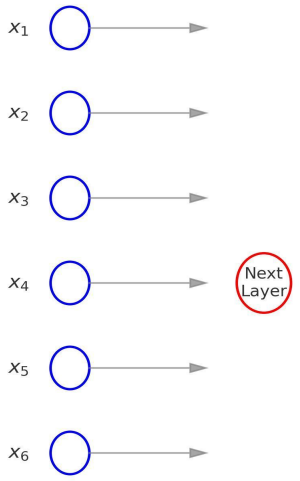
Output Layer: Produces the final prediction (e.g., class probabilities). Size depends on the task (e.g., 10 nodes for digit classification).



Input Layer

- **Entry point for the raw data** that the network will process.
- **Directly takes in the features or attributes** of each data sample. In a tabular dataset, each neuron could correspond to a specific column or feature.
- Number of neurons in the input layer is determined by the **dimensionality of the input data**.
- **Proper data preprocessing, like normalization or encoding,** is often applied to the input data.

Input Layer with d Neurons Feeding Next Layer



Here, d = 6 (number of input features)

Hidden Layer(s)

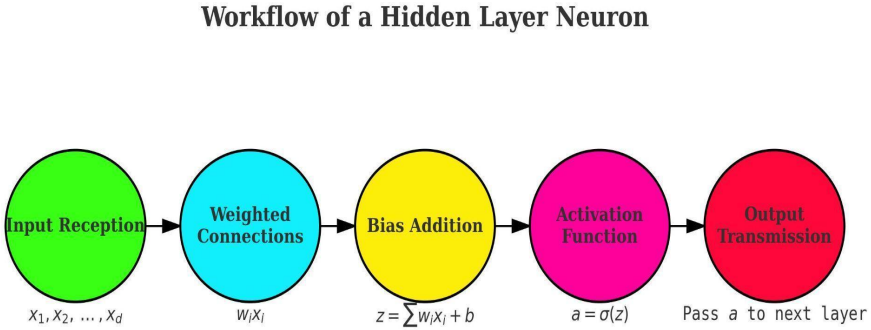
An **Intermediate processing** layer between the input and output layers that learns complex patterns and non-linear relationships in data by **transforming the inputs through weighted connections and non-linear activation functions**

Why is Hidden layer important ?

1. Learning Complexity : to learn and model complex, non-linear relationships in data

1. Feature Extraction : extract increasingly intricate features from the input data

1. Improved Function : By transforming data and identifying non-linear patterns, hidden layers significantly improve the network's ability to perform complex tasks such as classification, regression, and feature recognition.



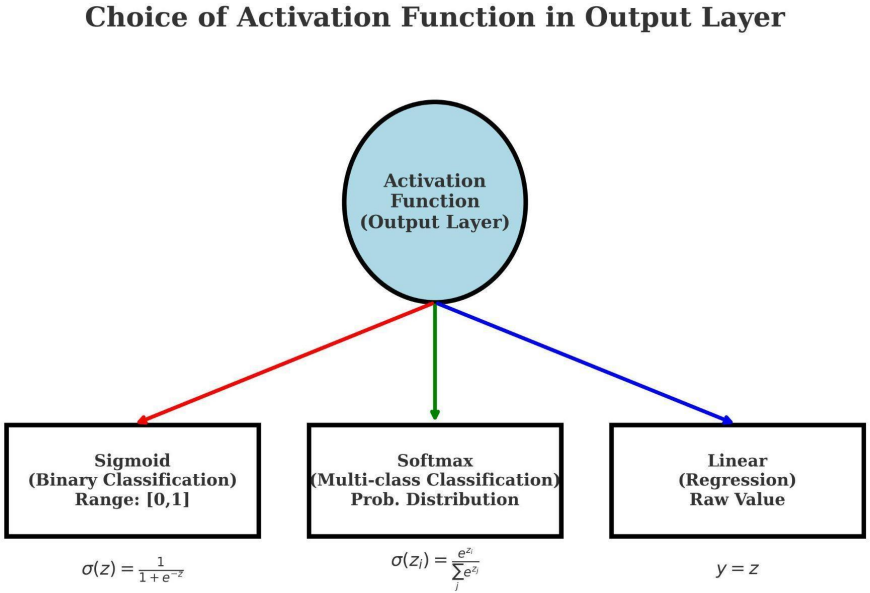
$$a = \sigma\left(\sum_{i=1}^{d} w_i x_i + b\right)$$

where:
- x_i: input features
- w_i: weights
- b: bias
- z: weighted sum ($\sum w_i x_i + b$)
- σ: activation function
- a: output of the neuron

Output Layer

- **Final layer** that produces predictions or classifications

- The number of nodes (neurons) in the output layer depends on the problem type:
  1. Binary Classification
  2. Multi- class classification
  3. Regression



The Crucial Role of Non-Linearity

Newton School of Technology

The Linear Trap

If we stack multiple layers of *linear* neurons, the result is still just a linear model.

$$y = W_2 (W_1 x) = (W_2 W_1) x = W_{new} x$$

Mathematically, a composition of linear functions is always linear. Depth without non-linearity is useless.

The Non-Linear Key

We must inject a non-linear **activation function** (sigma) after each layer.

$$y = W_2 (\sigma (W_1 x))$$

This allows the network to bend and twist the decision boundary, enabling it to approximate *any* function (Universal Approximation Theorem).

Function approximation f(x) = y with 3 hidden layers.

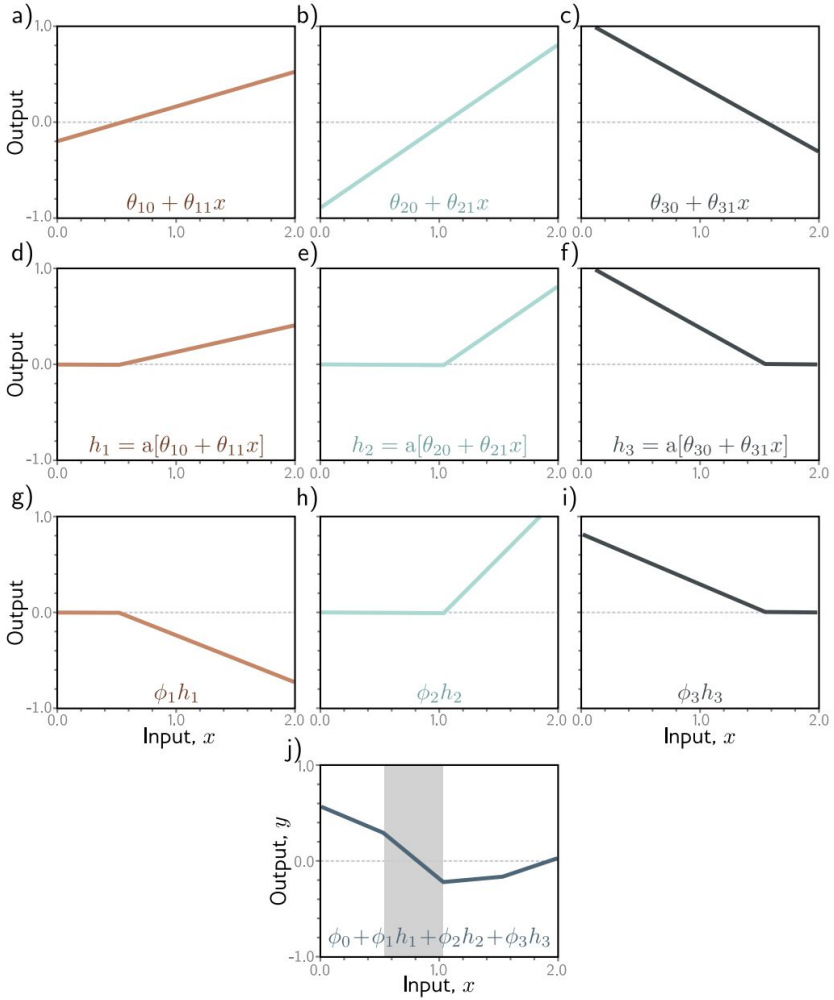
$$\begin{aligned} y &= \mathrm{f}[x, \boldsymbol{\phi}] \\ &= \phi_0 + \phi_1 \mathrm{a}[\theta_{10} + \theta_{11} x] + \phi_2 \mathrm{a}[\theta_{20} + \theta_{21} x] + \phi_3 \mathrm{a}[\theta_{30} + \theta_{31} x]. \end{aligned}$$

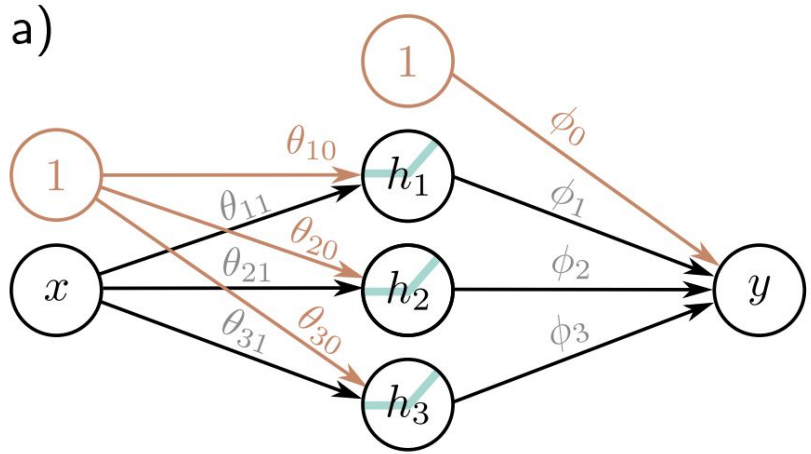
Where a is a chosen activation function, in this case:

$$\mathrm{a}[z] = \mathrm{ReLU}[z] = \begin{cases} 0 & z < 0 \\ z & z \geq 0 \end{cases}.$$

Geometry of Shallow NNs

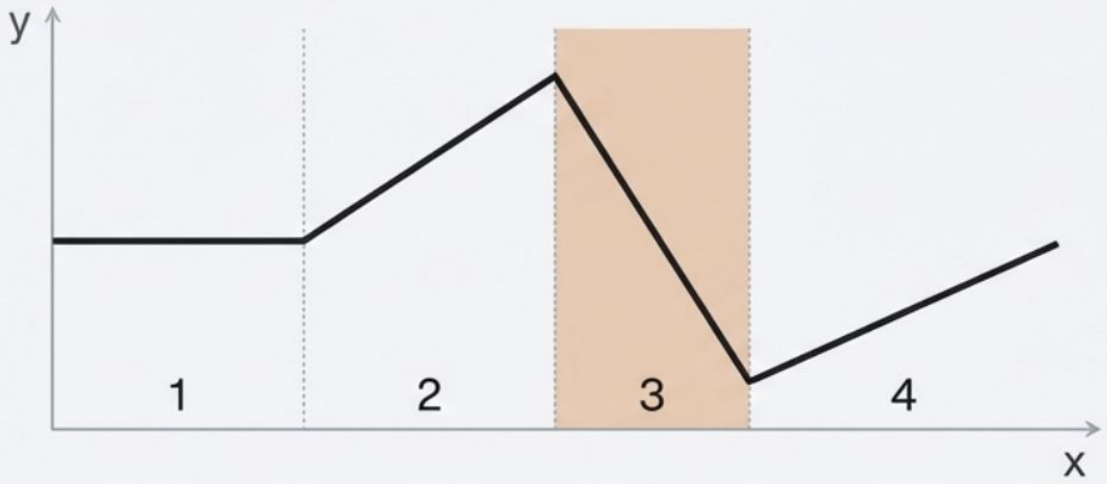
https://udlbook.github.io/udlfigures/





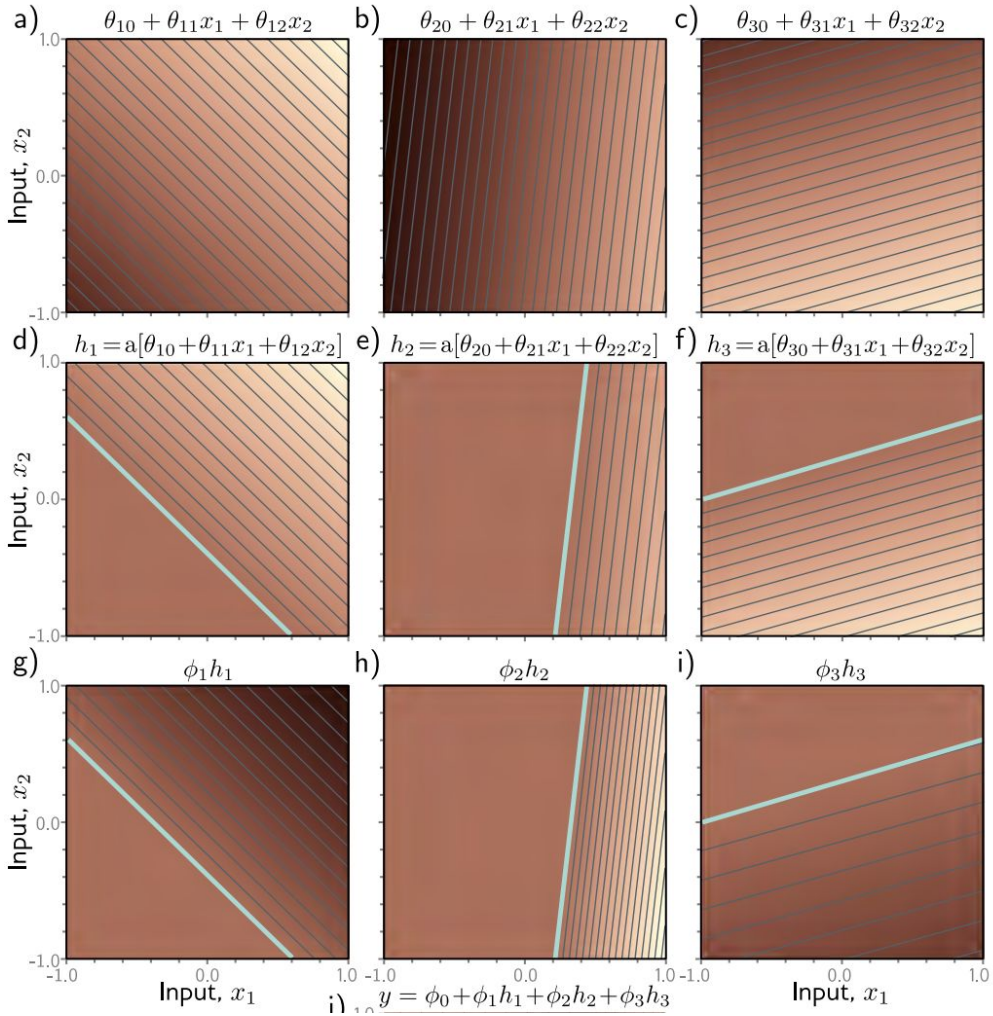
Problem 3.2 For each of the four linear regions in figure, indicate which hidden units are inactive and which are active

Problem 3.2



Region	h1	h2	h3
1	Inactive	Inactive	Inactive
2	Active	Inactive	Inactive
3 (Shaded)	**Active**	**Inactive**	**Active**
4	Active	Active	Active

"Active" means input > 0.
"Inactive" means clipped to 0.


a) $\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2$
b) $\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2$
c) $\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2$
d) $h_1 = \mathrm{a}[\theta_{10} + \theta_{11}x_1 + \theta_{12}x_2]$
e) $h_2 = \mathrm{a}[\theta_{20} + \theta_{21}x_1 + \theta_{22}x_2]$
f) $h_3 = \mathrm{a}[\theta_{30} + \theta_{31}x_1 + \theta_{32}x_2]$
g) $\phi_1 h_1$
h) $\phi_2 h_2$
i) $\phi_3 h_3$
Input, x_2
Input, x_1
1.0
0.0
-1.0
$y = \phi_0 + \phi_1 h_1 + \phi_2 h_2 + \phi_3 h_3$

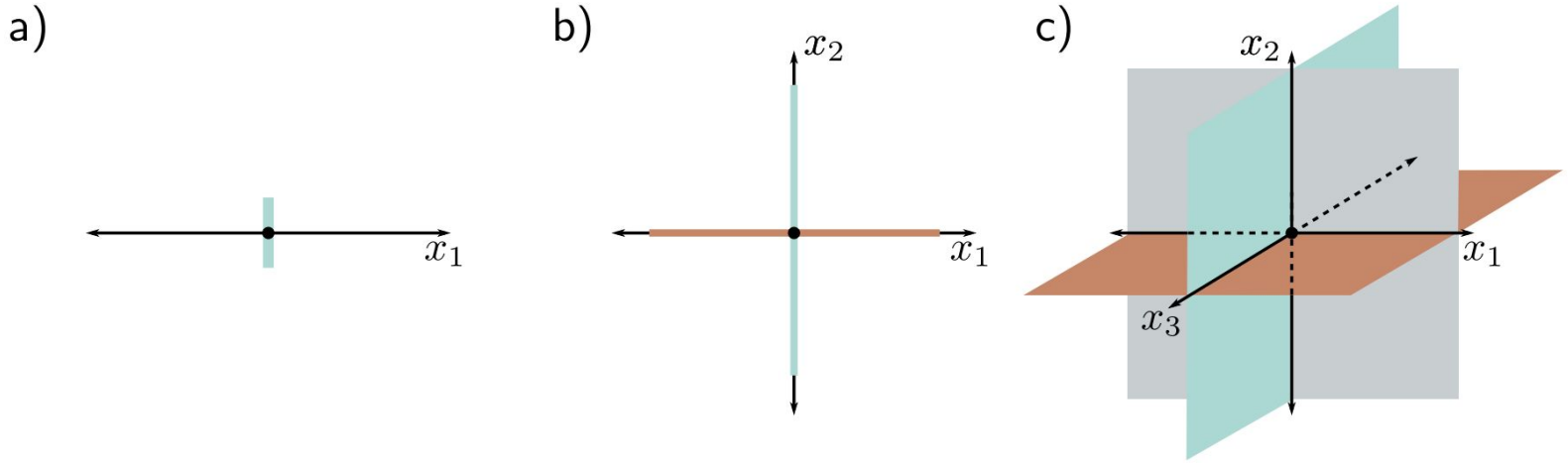


Figure 3.10 Number of linear regions vs. input dimensions. a) With a single input dimension, a model with one hidden unit creates one joint, which divides the axis into two linear regions. b) With two input dimensions, a model with two hidden units can divide the input space using two lines (here aligned with axes) to create four regions. c) With three input dimensions, a model with three hidden units can divide the input space using three planes (again aligned with axes) to create eight regions. Continuing this argument, it follows that a model with D_i input dimensions and D_i hidden units can divide the input space with D_i hyperplanes to create 2^{D_i} linear regions.

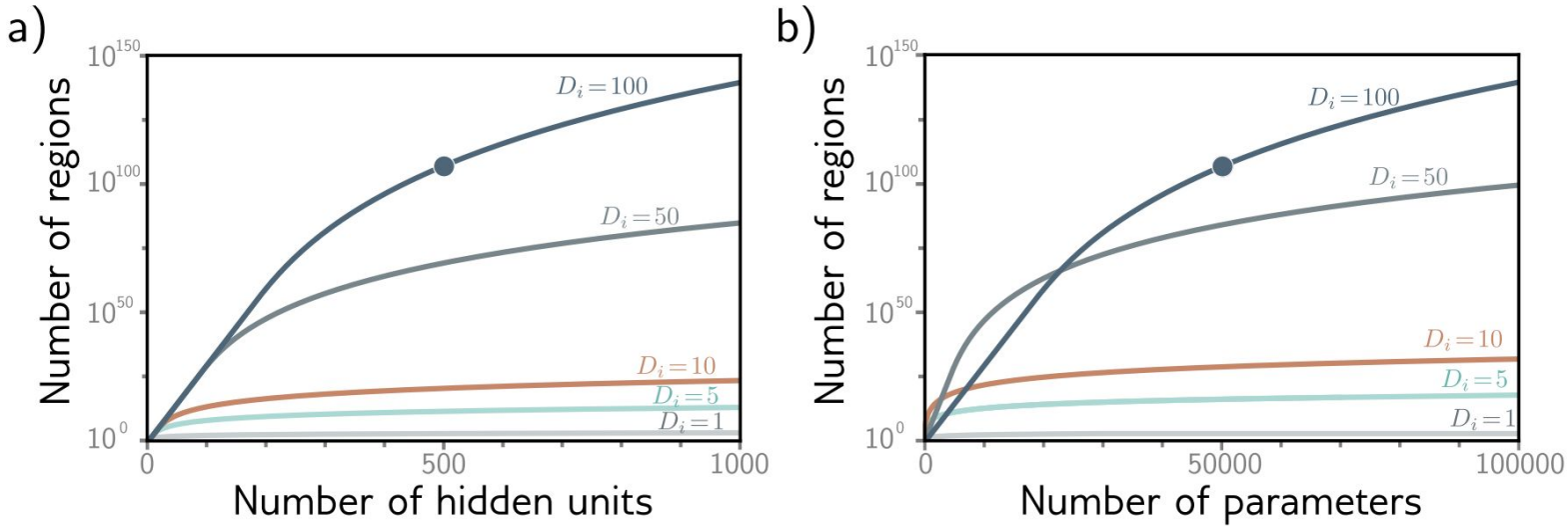


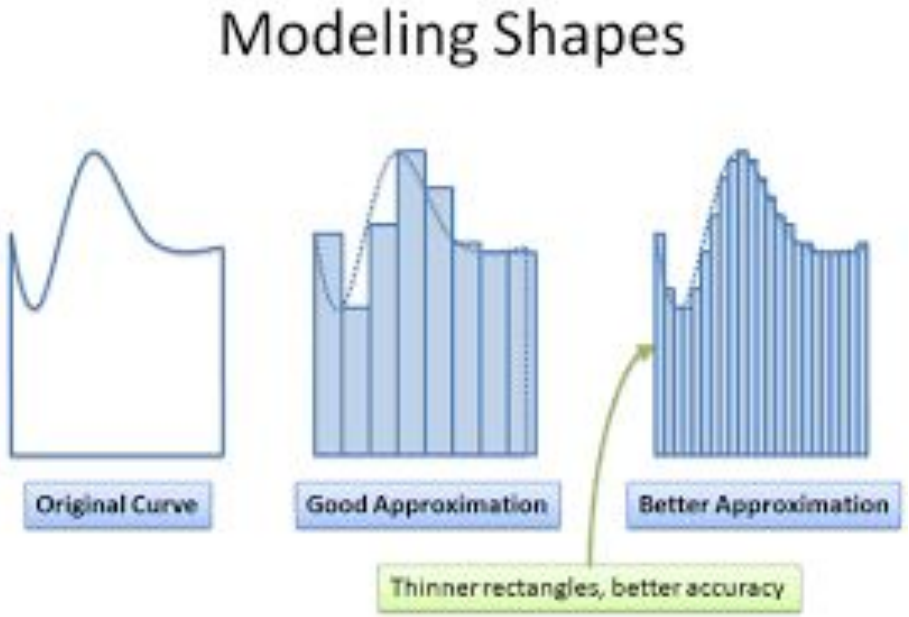
Figure 3.9 Linear regions vs. hidden units. a) Maximum possible regions as a function of the number of hidden units for five different input dimensions $D_i = \{1, 5, 10, 50, 100\}$. The number of regions increases rapidly in high dimensions; with $D = 500$ units and input size $D_i = 100$, there can be greater than 10^{107} regions (solid circle). b) The same data are plotted as a function of the number of parameters. The solid circle represents the same model as in panel (a) with $D = 500$ hidden units. This network has $51,001$ parameters and would be considered very small by modern standards.

The number of hidden units in a shallow network is a measure of the *network capacity*. With ReLU activation functions, the output of a network with D hidden units has at most D joints and so is a piecewise linear function with at most $D+1$ linear regions. As we add more hidden units, the model can approximate more complex functions.

Indeed, with enough capacity (hidden units), a shallow network can describe any continuous 1D function defined on a compact subset of the real line to arbitrary precision.

Same notion as calculus

Break a curve into small rectangles



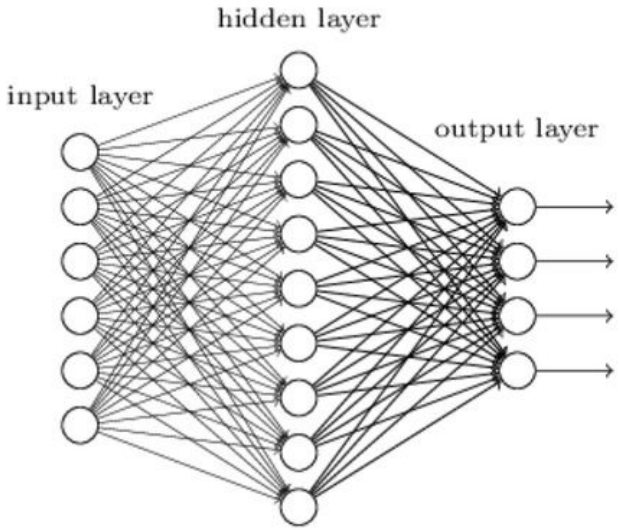
Universal Approximation Theorem

The Theorem

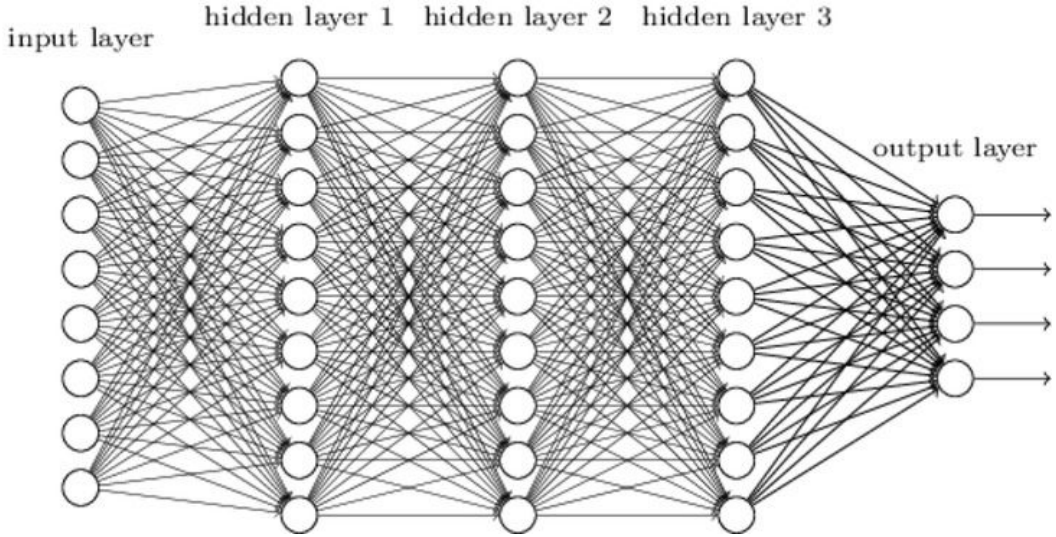
A feed-forward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of Rn.

The Catch: To approximate complex functions, that single layer might need an *exponentially* large number of neurons (infinite width).

"Non-deep" feedforward neural network



Deep neural network



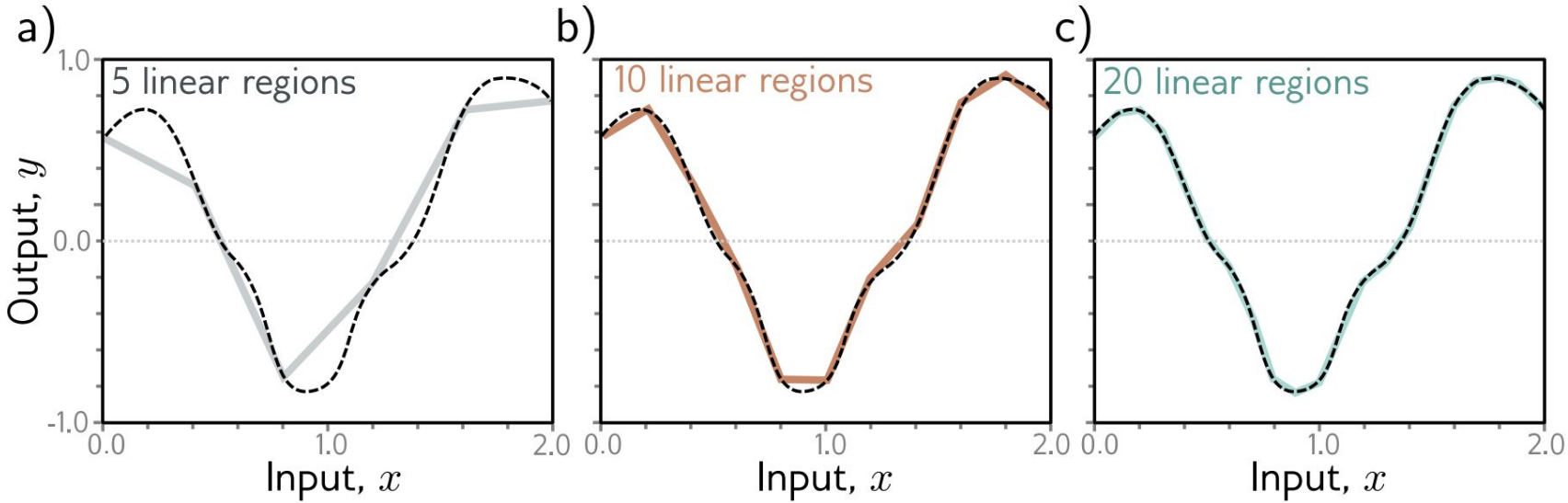


Figure 3.5 Approximation of a 1D function (dashed line) by a piecewise linear model. a–c) As the number of regions increases, the model becomes closer and closer to the continuous function. A neural network with a scalar input creates one extra linear region per hidden unit. This idea generalizes to functions in D_i dimensions. The universal approximation theorem proves that, with enough hidden units, there exists a shallow neural network that can describe any given continuous function defined on a compact subset of $\mathbb{R}^{D_i}$ to arbitrary precision.

Choosing the Activation Function

Why do we need Activation Functions?

Without a non-linear activation function, a Neural Network is just a sequence of linear transformations.

Non-linear functions allow neural networks to form curved decision boundaries, making them capable of handling complex patterns (e.g., classifying apples vs. bananas under varying colors and shapes).

No matter how deep, a linear network can only approximate linear functions. Activation functions introduce non-linearity, allowing the network to learn complex, curved decision boundaries.

Activation Functions

Newton School of Technology

Activation functions enable networks to learn complex, non-linear mappings between inputs and outputs.

Activation Function	Formula	Range	Key Features
Sigmoid	$\frac{1}{1+e^{-z}}$	(0, 1)	Outputs probabilities; suffers from vanishing gradients.
Tanh	$\frac{e^{z}-e^{-z}}{e^{z}+e^{-z}}$	(-1, 1)	Centered around zero; suffers from vanishing gradients but less than Sigmoid.
ReLU	$\max(0, z)$	[0, ∞]	Simple and efficient; can suffer from "dying ReLU."
Leaky ReLU	$\max(\alpha z, z)$	(-∞, ∞)	Solves the "dying ReLU" problem by allowing small gradients for negative values.
Softmax	$\frac{e^{z_i}}{\sum_j e^{z_j}}$	(0, 1)	Converts logits into probabilities for multi-class classification.
ELU	$\begin{cases} z & z > 0 \\ \alpha(e^{z}-1) & z \leq 0 \end{cases}$	(-α, ∞)	Combines the benefits of ReLU and Tanh, with faster convergence.

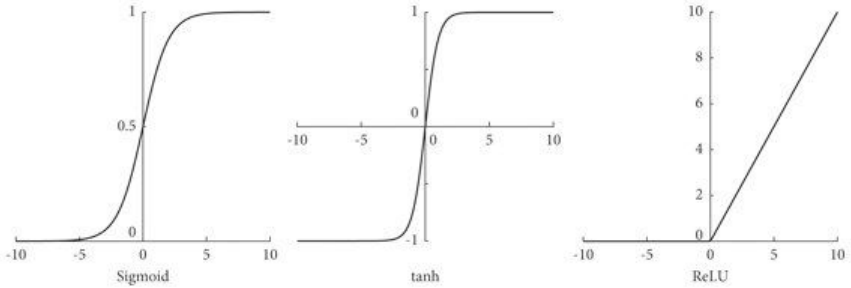
Activation Functions of the past: Sigmoid & Tanh

Newton School of Technology

The S-Curves

Sigmoid (sigma): Squeezes output between 0 and 1. Historically used, but suffers from "vanishing gradients."

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$



Tanh: Squeezes output between -1 and 1.

Zero-centered, which is often better for optimization than Sigmoid.

$$f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

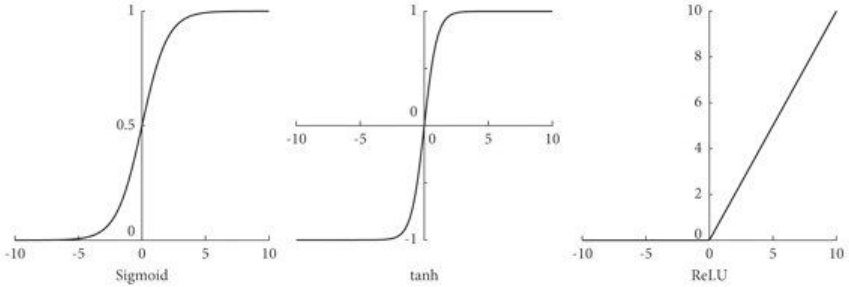
Activation Function (most relevant) : ReLU

Newton School of Technology

Rectified Linear Unit (ReLU)

$$f(x) = \begin{cases} x, & \text{if } x \geq 0 \\ \alpha x, & \text{if } x < 0 \end{cases}$$

The modern default for hidden layers.



Efficiency: Computationally almost free (just a threshold check).

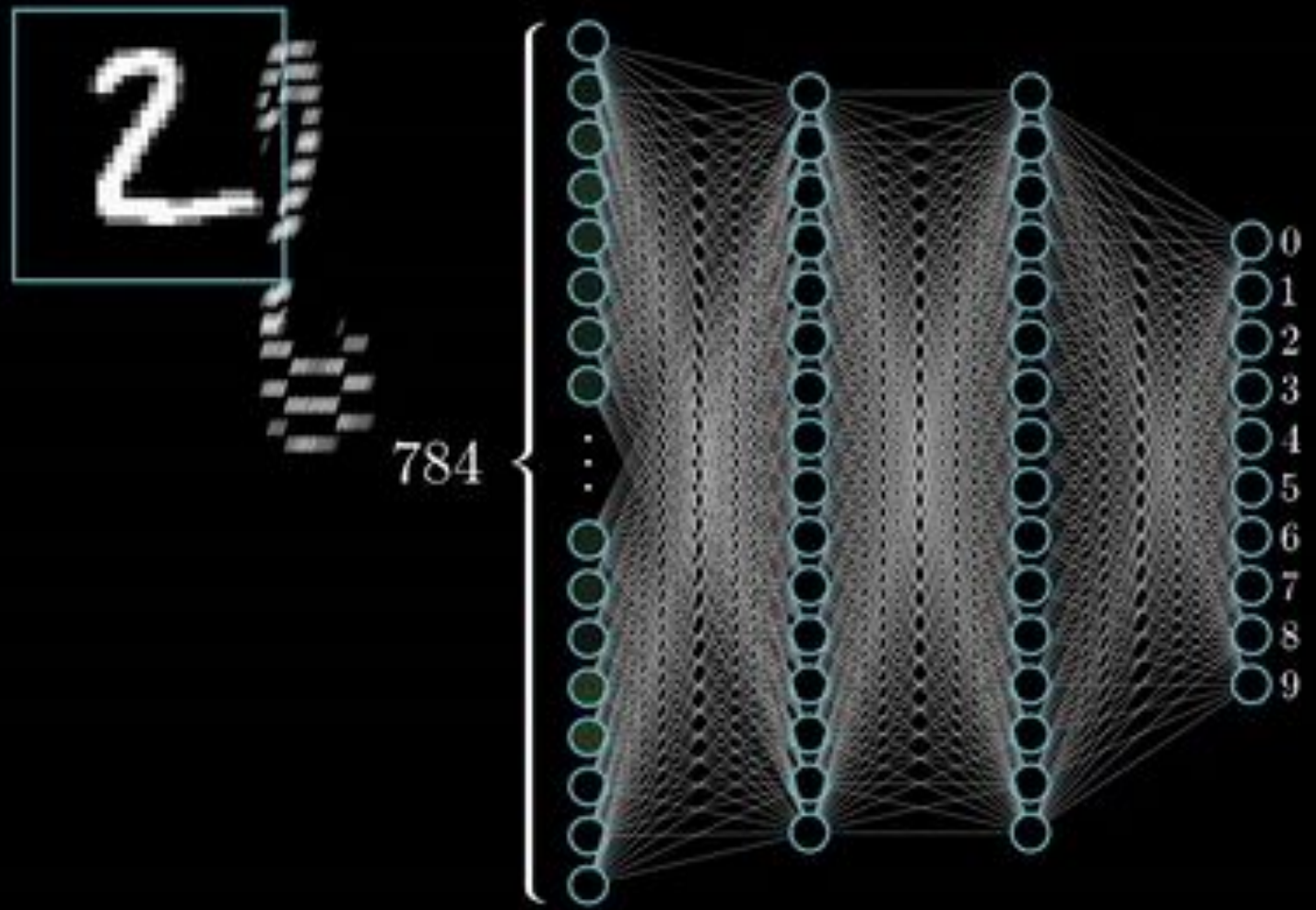
Sparsity: Outputs true zero for negative inputs.

No Vanishing Gradient: For positive inputs, the gradient is constant (1), allowing fast learning in deep networks.

Problem 3.8 Consider replacing the ReLU activation function with (i) the Heaviside step function heaviside$[z]$, (ii) the hyperbolic tangent function $\tanh[z]$, and (iii) the rectangular function rect$[z]$, where:

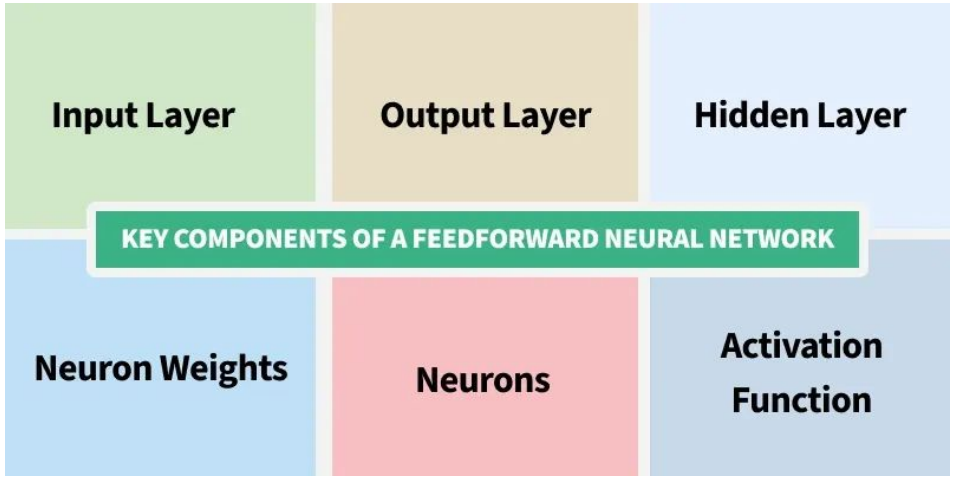
$$\text{heaviside}[z] = \begin{cases} 0 & z < 0 \\ 1 & z \geq 0 \end{cases} \qquad \text{rect}[z] = \begin{cases} 0 & z < 0 \\ 1 & 0 \leq z \leq 1 \\ 0 & z > 1 \end{cases} . \tag{3.15}$$

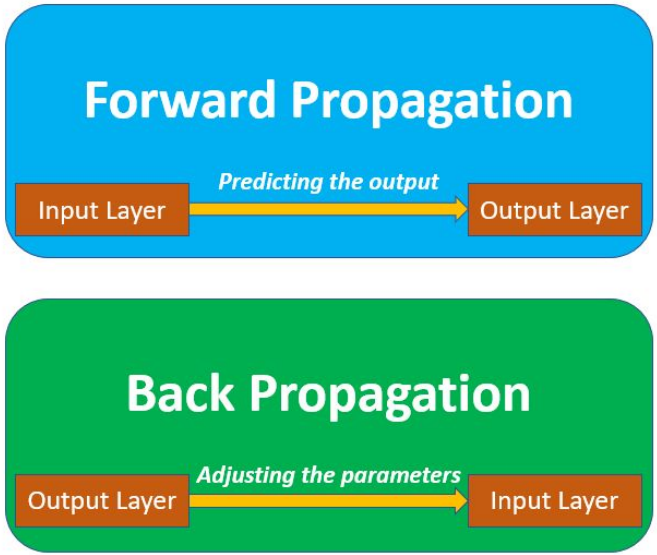
Inference : Using the model


784
0
1
2
3
4
5
6
7
8
9

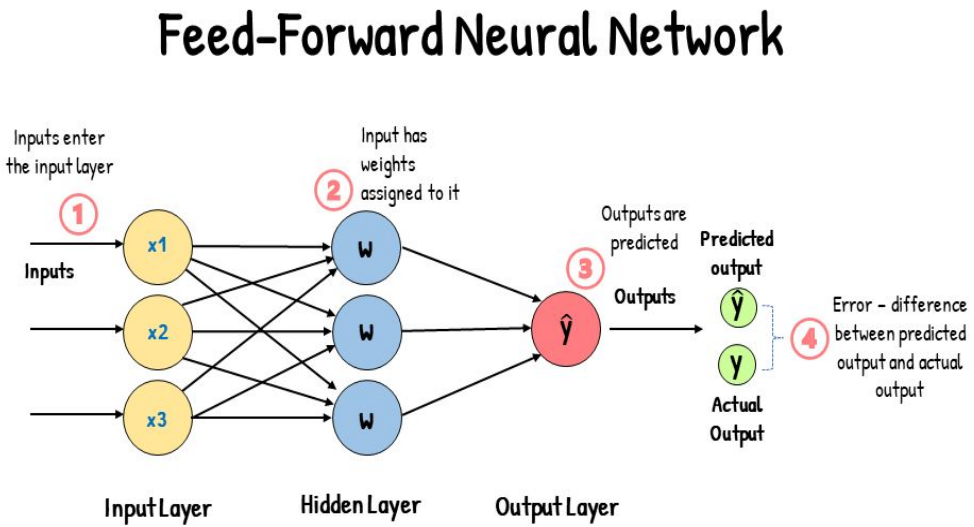
Forward Pass in Neural Network

How Data Flows Through the Network ?





Forward Pass in Neural Network



At each layer:

- Linear transformation: z=Wx+b
- Non-linear activation: a=n(z)
- Pass a to the next layer.

Input Layer:

- Raw data is fed into the network.

Hidden Layers:

- The data is processed through several layers of neurons. Each neuron performs a calculation using its inputs, weights (representing connection strength), and a bias, then passes the result through an activation function.

Output Layer:

- The processed information reaches the final layer, where the network generates its prediction or classification.

Forward Propagation

Mathematical Steps

1. **Weighted Sum Calculation:**

$$z = \sum_{i=1}^{n} w_i x_i + b$$

2. **Activation Function Application:**

$$y = \phi(z)$$

Example: Predicting Loan Approval

Inputs:

- x_1: Credit Score (normalized between 0 and 1)
- x_2: Income Level (normalized between 0 and 1)

Weights and Bias:

- $w_1 = 0.7$ (credit score is important)
- $w_2 = 0.3$ (income level is less important)
- $b = -0.5$ (bias towards not approving)

Activation Function: Sigmoid

Calculation:

For an applicant with a credit score of 0.8 and income level of 0.6:

$$\begin{aligned} z &= (0.7)(0.8) + (0.3)(0.6) - 0.5 \\ &= 0.56 + 0.18 - 0.5 = 0.24 \\ y &= \frac{1}{1 + e^{-0.24}} \approx 0.560 \end{aligned}$$

Interpretation: There's a 56% chance of loan approval based on the model.

Vectorization & Matrix Operations

Newton School of Technology

Efficiency through Matrices

We don't compute loops for each neuron. We use Linear Algebra to compute entire layers at once.

- X Input vector (or batch matrix).
- W: Weight matrix connecting layers.
- Z: Pre-activation linear output.
- A: Post-activation non-linear output.

Forward Propagation (Vectorized)

Instead of looping, we use Linear Algebra (Matrix Multiplication) for efficiency.

Layer 1

$$Z^{[1]} = W^{[1]} X + b^{[1]}$$

$$A^{[1]} = \sigma(Z^{[1]})$$

Layer 2 (Output)

$$Z^{[2]} = W^{[2]} A^{[1]} + b^{[2]}$$

$$\hat{y} = A^{[2]} = \sigma(Z^{[2]})$$

The Final Output & Loss Function

Newton School of Technology

The Prediction

The final layer produces the model's prediction.

Binary Classification: Sigmoid unit (probability 0-1).

Multi-Class: Softmax layer (probability distribution).

Regression: Linear unit (continuous value).

The Cost Function

How do we know if the prediction is good?

We compare prediction with ground truth.

Loss Function: Measures the error (e.g., Mean Squared Error, Cross-Entropy).

Goal: Minimize this cost J by adjusting weights (Training).

Training : Building the model

Context: The Neural Network as a Function

At its core, a neural network is simply a parametric function f that maps an input x to an output y.

$$y = f(x; \theta)$$

Where **θ** represents the parameters (weights and biases) of the network.

Goal: Find the optimal θ such that the predicted y matches the true targets.

Key Components

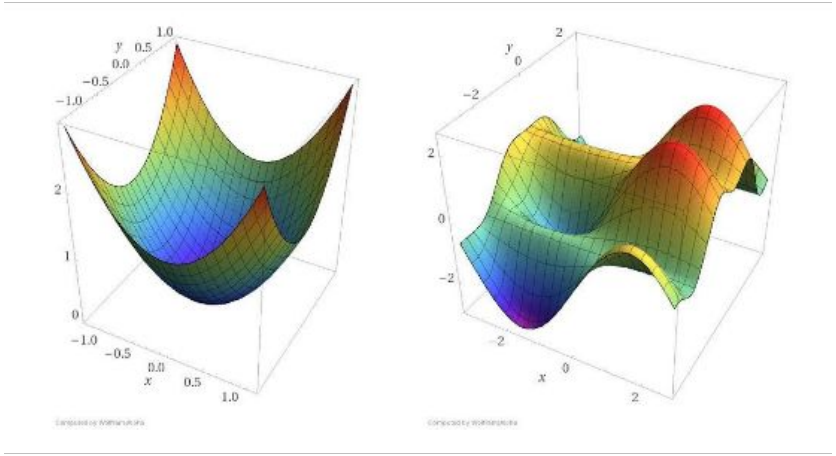
- **Architecture:** The structure of the function (layers, neurons).
- **Loss Function:** The metric of error.
- **Optimization:** The method to update θ.

The Objective: Loss Minimization

We define a Loss Function, J(θ), which quantifies the error between predictions and ground truth.

The training process is an optimization problem: finding the global minimum in a high-dimensional, non-convex landscape.

$$\theta^* = \underset{\theta}{\operatorname{argmin}} J(\theta)$$



Visualizing the Loss Landscape

Imagine the Loss Function J as a geographical landscape (valleys and hills).

The "altitude" is the Error.

Our goal is to reach the lowest point (Global Minimum).

Problem: We are dropped at a random location (random weight initialization) and it's pitch dark.